

b) Investigación

HTTP:

- ¿Qué es HTTP y como funciona? ¿Cuáles son los métodos de petición y que realiza cada uno? ¿Qué diferencia existen entre una Petición GET y POST? ¿Cómo es el envío de parámetros, qué significa “=” y que significa “&” cuando se envían parámetros por POST y GET? ¿Cuáles son los códigos de estado en HTTP?

HTTP (HyperText Transfer Protocol) es un protocolo de comunicación con el que diferentes partes de internet se comunican entre sí. Es un lenguaje basado en el modelo cliente - servidor y en un esquema de “solicitud - respuesta”: el cliente pide un recurso y el servidor responde. Es un protocolo sin estado: cada petición es independiente y el servidor no recuerda las anteriores.

Métodos de petición y qué hacen :

Método	Qué hace
GET	Obtiene un recurso. No modifica nada en el servidor
POST	Envía datos para crear un recurso o ejecutar una acción
PUT	Reemplaza un recurso completo (o lo crea si no existe)
PATCH	Modifica parcialmente un recurso
DELETE	Elimina un recurso
HEAD	Igual que GET, pero devuelve solo los encabezados, sin cuerpo
OPTIONS	Consulta qué métodos admite un recurso (también lo usan los navegadores para CORS)
CONNECT	Abre un túnel hacia el servidor (se usa con proxies y HTTPS)
TRACE	Devuelve la petición tal como llegó, para diagnóstico

Diferencia entre GET y POST:

La diferencia entre el get y post además de para qué son utilizados es cómo se

envían los datos, en una petición get los parámetros se pasan por la url (quedando visibles en la barra, el historial y los logs), en cambio, en una petición post los parámetros se pasan en el cuerpo de la misma.

Envío de parámetros: = y &:

En una petición get los parámetros van al final de la URL, después de ?:

`https://sitio.com/buscar?nombre=Ana&edad=25`

En una petición post van en el cuerpo, generalmente con el mismo formato:

`nombre=Ana&edad=25`

Códigos de estado en HTTP:

Informativos:

100 Continue

101 Switching Protocols

Éxito:

200 OK

201 Created

202 Accepted

204 No content

Redirección

301 Moved Permanently

302 Found

304 Not Modified

Error del cliente

400 Bad Request

401 Unauthorized

403 Forbidden,

404 Not Found,

405 Method Not Allowed,

409 Conflict,

429 Too Many Requests

Error del servidor:

500 Internal Server Error

502 Bad Gateway

503 Service Unavailable,

504 Gateway Timeout

REST – API:

- ¿Qué es REST y para que sirve? ¿Qué es una API, para que sirve y cual es el objetivo de combinarlo con REST? ¿Qué es un Endpoint?. Realizar un ejemplo Gráfico del Uso de API REST.

¿Qué es REST y para qué sirve?

REST (Representational State Transfer) es un estilo de arquitectura para diseñar servicios web que se apoya en HTTP. Cada cosa que expone el servidor es un recurso (personas, productos, pedidos) identificado por una URI, y el cliente lo manipula con los métodos HTTP (GET, POST, PUT, DELETE). Sirve para que sistemas distintos (un front en React, una app móvil, otro servidor) se comuniquen con un mismo backend de forma simple y estandarizada.

Principios de REST:

- Cliente-servidor: el front y el back están separados.
- Sin estado (stateless): cada petición lleva toda la información necesaria.
- Interfaz uniforme: recursos identificados por URI y manipulados con los verbos HTTP.
- Representaciones: el recurso viaja en un formato como JSON o XML.
- Cacheable: las respuestas pueden indicar si se pueden guardar en caché.
- Sistema en capas: puede haber proxies, balanceadores, etc. entre cliente y servidor.

¿Qué es una API, para qué sirve y cuál es el objetivo de combinarla con REST?

Una API (Application Programming Interface) es un conjunto de reglas y operaciones que un sistema expone para que otros programas puedan usarlo sin conocer su implementación interna. Sirve para reutilizar funcionalidad y separar responsabilidades.

Combinarla con REST permite que la API sea simple, predecible y use el protocolo que ya existe en toda la web (HTTP). Así cualquier cliente que “hable HTTP”

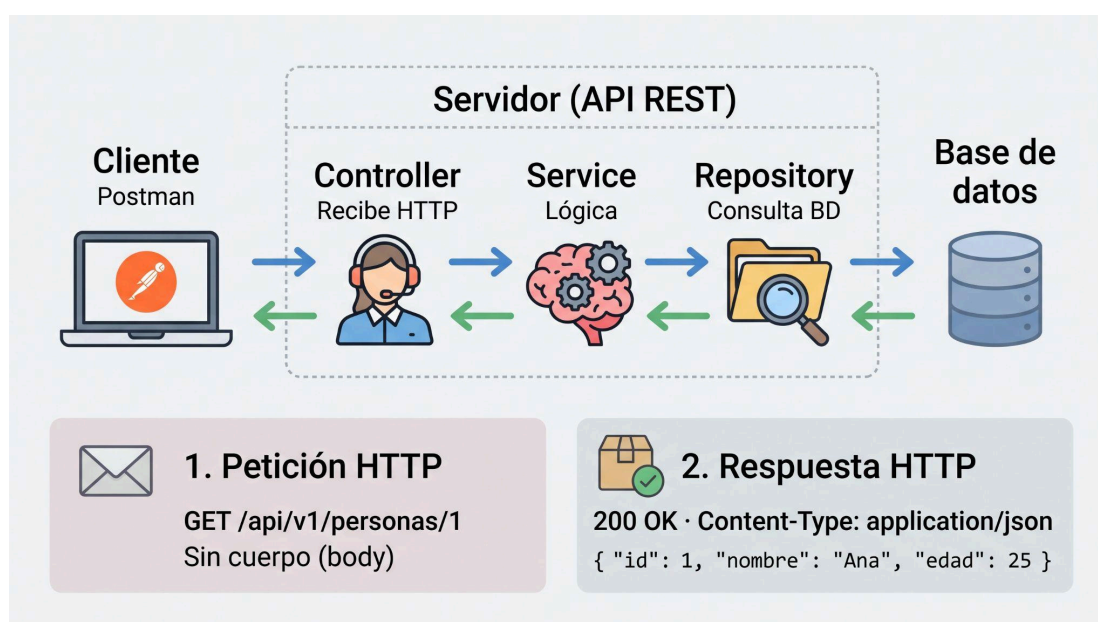
puede consumirla, sin importar el lenguaje o la tecnología con la que esté hecho.

¿Qué es un Endpoint?

Es cada URL concreta de la API a la que se puede hacer una petición, combinada con un método HTTP. Es el “punto de entrada” a una operación. Ejemplo:

Método	Endpoint	Acción
GET	/api/v1/personas	Listar todas
GET	/api/v1/personas/1	Obtener la persona 1
POST	/api/v1/personas	Crear una persona
PUT	/api/v1/personas/1	Actualizar la persona 1
DELETE	/api/v1/personas/1	Eliminar la persona 1

Ejemplo gráfico del uso de una API REST



Flujo: el cliente envía la petición (método + URI + cabeceras + cuerpo opcional) → el Controller la recibe → el Service aplica la lógica → el Repository consulta la base de datos → el servidor responde con un código de estado y el recurso en JSON.

- Mencione cuales son los patrones de diseño que se aplican para API. Realizar una explicación de cada uno de ellos.

Patrones de diseño que se aplican en una API

Patrón	Explicación
--------	-------------

MVC / capas (Controller – Service – Repository)	Separa la API en capas: el Controller recibe las peticiones HTTP, el Service contiene la lógica de negocio y el Repository accede a los datos. Cada capa tiene una única responsabilidad.
Repository / DAO	Encapsula el acceso a la base de datos detrás de una interfaz. El resto del sistema no sabe si los datos vienen de MySQL, Mongo u otra fuente. En Spring es JpaRepository.
DTO (Data Transfer Object)	Objeto simple que se usa para enviar y recibir datos por la API en lugar de exponer directamente las entidades. Evita mostrar campos sensibles (ej. contraseña) y desacopla la API de la base de datos.
Inyección de dependencias (IoC)	Las clases no crean sus dependencias, se las “inyecta” el framework. Facilita los tests y reduce el acoplamiento (@Autowired / inyección por constructor en Spring).
Singleton	Garantiza una única instancia de una clase. Los beans de Spring (Services, Controllers) son singleton por defecto.
Builder	Construye objetos complejos paso a paso, de forma legible (ej. armar una respuesta o un DTO con muchos campos).
Adapter / Mapper	Convierte un objeto en otro (entidad ↔ DTO), adaptando la representación interna a la que expone la API.
Facade / API Gateway	Ofrece un único punto de entrada simple que oculta la complejidad de varios servicios internos. Es muy usado en arquitecturas de microservicios.

JSON:

- ¿Qué es JSON y para qué sirve? ¿Cómo es el formato de JSON? Dar un ejemplo de un objeto y de una colección de Objetos en Java y su equivalente en JSON. ¿Cuáles alternativas existen además de JSON?

¿Qué es JSON y para qué sirve?

JSON (JavaScript Object Notation) es un formato de texto liviano para representar e intercambiar datos estructurados. Es legible para las personas y fácil de procesar para las máquinas. Sirve para transportar información entre cliente y servidor en las APIs REST (es el formato más usado), y también para archivos de

configuración.

¿Cómo es el formato de JSON?

- Un objeto va entre llaves { } y contiene pares "clave": valor, separados por comas.
- Una colección (array) va entre corchetes [], con los elementos separados por comas.
- Las claves siempre van entre comillas dobles.
- Los valores pueden ser: texto ("Ana"), número (25), booleano (true / false), null, otro objeto o un array.

Ejemplo de un objeto en Java y su equivalente en JSON

```
1 //Java
2
3 public class Persona {
4     private Long id;
5     private String nombre;
6     private int edad;
7     private boolean activo;
8     // constructor, getters y setters
9 }
10
11 Persona p = new Persona(1L, "Ana", 25, true);
12
```

```
1
2
3 {
4     "id": 1,
5     "nombre": "Ana",
6     "edad": 25,
7     "activo": true
8 }
9
10
```

Ejemplo de una colección de objetos en Java y su equivalente en JSON

```

1 //Java
2
3 List<Persona> personas = new ArrayList<>();
4 personas.add(new Persona(1L, "Ana", 25, true));
5 personas.add(new Persona(2L, "Luis", 30, false));
6
7

```

```

1 [
2   {
3     "id": 1,
4     "nombre": "Ana",
5     "edad": 25,
6     "activo": true
7   },
8   {
9     "id": 2,
10    "nombre": "Luis",
11    "edad": 30,
12    "activo": false
13  }
14 ]

```

¿Qué alternativas existen además de JSON?

Formato	Descripción
XML	Formato basado en etiquetas (<persona><nombre>Ana</nombre></persona>). Es más verboso, muy usado en servicios SOAP y sistemas antiguos.
YAML	Basado en indentación, muy legible. Se usa sobre todo en configuración (Docker Compose, application.yml de Spring).
CSV	Datos tabulares separados por comas. Simple, pero no soporta estructuras anidadas.
Protocol Buffers (Protobuf)	Formato binario de Google, muy compacto y rápido. Se usa en gRPC.
MessagePack / BSON	Versiones binarias tipo JSON, más compactas (BSON es el que usa MongoDB).
TOML	Formato de configuración simple y legible (ej.

	pyproject.toml).
--	------------------

POSTMAN:

- ¿Qué es Postman? ¿Cuál es la dirección de su sitio oficial? Dar un ejemplo cómo utilizar GET, POST, PUT, DELETE.

Es una herramienta para probar el funcionamiento de nuestras APIs. Permite armar peticiones HTTP, enviarlas a un servidor y ver la respuesta sin necesidad de tener que programar un cliente. Se usa para probar el servidor antes de conectarla con un front.

Sitio oficial: <https://www.postman.com>

Ejemplo:

- GET URI (<http://localhost:8080/api/v1/personas>)

No lleva cuerpo. La respuesta es un 200 OK con el JSON de las personas.

- POST: crear un recurso

POST URI (<http://localhost:8080/api/v1/personas>)

En Body → raw → JSON agregas el cuerpo de la petición

- PUT URI (<http://localhost:8080/api/v1/personas/1>)

Ponemos el cuerpo en formato JSON también

- DELETE URI (<http://localhost:8080/api/v1/personas/1>)

No lleva cuerpo. La respuesta es 204 No Content. Si repetís el pedido con un id que ya no existe, el servidor responde con un error.